H-OS

— An AI Operating System for H-FARM

Project definition | 24 October 2026
Status: Proposed architecture; vertical slice to be implemented and validated.
Scope: Open-source prototype using synthetic or public data and simulated actions.

1. Vision

H-OS is an AI operating layer for a campus organization. Its purpose is to connect fragmented information, understand operational relationships, detect relevant changes, and turn them into **traceable actions**.

The long-term ambition is a shared organizational model that can support campus operations, events, education, and business processes without rebuilding the same integrations and controls for every application.

Core principle: Information → Organizational state → Decision → Authorized action → Feedback.

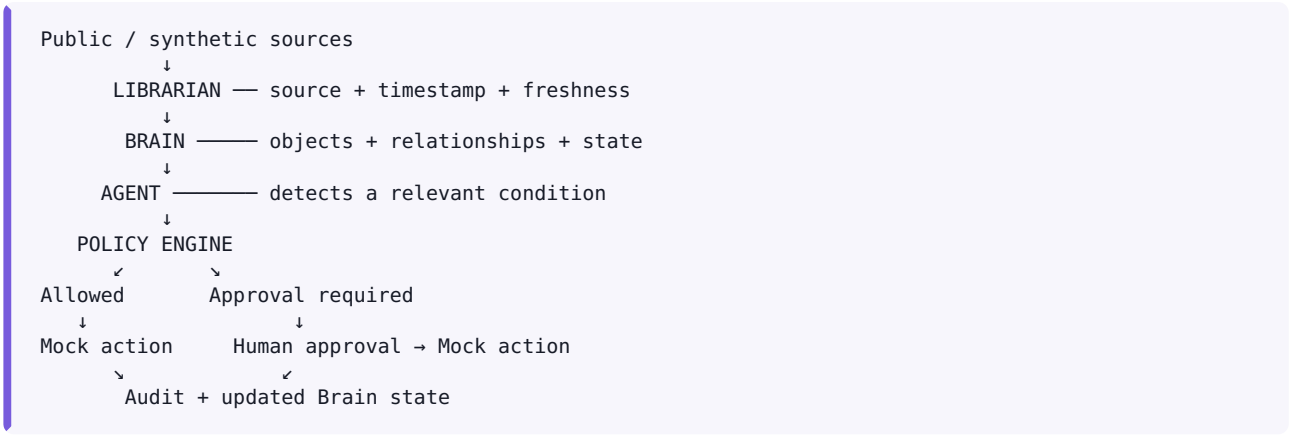
H-OS is inspired by the *operational ontology* approach of Palantir Foundry, but it is not a clone of Foundry or an actual operating system kernel. The prototype proves one repeatable workflow, not an entire enterprise platform.

2. The theoretical model

H-OS has three main components, matching the H-FARM challenge brief:

Component	Responsibility	Output
Librarian	Ingest documents and events; extract facts; preserve provenance; detect changes and stale records.	Reliable, versioned facts with source references.
Brain	Represent organizational objects, their relationships, current state, role-based visibility, and available actions.	A shared operational model.
Agents	Observe state changes; propose or perform permitted actions; request human approval when policy requires it.	Controlled actions and audit records.

Cross-cutting controls: identity and role permissions, deterministic safety policies, timestamps, source references, and execution logs.



The ontology: the shared language

Instead of storing disconnected documents, the Brain models **objects, relations, and actions**. A minimal campus ontology includes:

- **Objects:** Event, Room, Booking, Device, SensorReading, Source, Action.
- **Relationships:** Event *uses* Room; Booking *reserves* Room; Device *belongs to* Room; Reading *describes* Device; Action *affects* Device.
- **Actions:** release booking, switch mock lights off, request boiler adjustment, approve or reject a request.

Each relevant fact retains a source ID, observation time, and freshness status. A statement without sufficient evidence must be marked **unknown**, not invented.

The model is **operational** because changing an object's state can trigger a governed workflow. It is **extensible** because future applications can reuse objects, policies, and actions.

3. First vertical slice: Campus Event Operations

Scenario: A scheduled campus event is canceled. Its room becomes unbooked, but simulated sensors still report energy consumption.

1. **Ingest:** The Librarian reads a synthetic event update, booking records, and mock device readings.
2. **Understand:** Gemma 4 E2B extracts a proposed event status change and supporting source references. The application validates the structured output.
3. **Update:** The Brain updates the Event and Booking state, retaining provenance and previous versions.
4. **Detect:** A proactive agent detects: no active booking + no occupancy + lights on.
5. **Act:** A deterministic policy permits switching **mock lights** off. The system records this automatically.
6. **Escalate:** Boiler changes require explicit Facility Manager approval before any mock action.
7. **Explain:** The interface displays the decision, sources, policy, actor, timestamps, and resulting state.

Three roles: Event Organizer (event/booking information), Facility Manager (facility readings and operational approvals), Administrator (system-wide oversight and audit). All role checks are enforced on the backend, not only in the interface.

4. Gemma 4: precise responsibility

The vertical slice uses **Gemma 4 E2B**, an open-weight model that can run locally via Ollama. Gemma handles **ambiguous language**, such as interpreting a cancellation notice; it does **not** decide access rights, safety policies, or device execution.

Model input: an event notice and its source metadata.
Expected structured output: `event_id`, `new_status`, `effective_at`, `source_id`, `evidence`, `uncertainty`.

Application checks: valid JSON/schema; known event and source IDs; evidence actually present in the source; no contradictions with authoritative records. Invalid or uncertain extraction goes to human review; the agent must not act on it.

Local inference: `ollama run gemma4:e2b`. The prototype should compare Gemma against a deterministic parser on the **same test notices**, recording extraction correctness, latency, and human corrections. If the model does not improve the total process, retain the simpler parser.

5. Minimal implementation

Layer	Minimal choice
Interface	One web dashboard with views for three roles
Backend	One service with typed API and policy checks
Brain	SQLite or PostgreSQL with explicit objects, relationships, and action history
Librarian	Synthetic JSON/CSV documents, revision detection, source tracking
AI	Gemma 4 E2B through local Ollama
Agent	Event-driven or polling rule that reacts to Brain state changes
Actions	Mock device state changes only

No production integrations, complex multi-agent framework, vector database, distributed knowledge graph, or real building automation are necessary for the first demonstration. MCP can be added later if external agents need a standard tool interface.

6. What the demo must prove

A single live loop must show:

Source updated → Librarian detects change → Brain updates → Agent reacts → mock lights turn off → boiler approval appears → authorized user approves → all results and sources remain visible.

A small set of tests should also show that the system:

- Does not act when a booking is still active, occupancy is nonzero, or evidence is missing/stale.
- Does not execute approval-required actions without authorization.
- Does not reveal restricted data to an unauthorized role.
- Preserves the original source and an immutable action history.

Challenge deliverables: open-source repository under MIT or Apache-2.0, functioning Librarian and Brain, one proactive agent, three role-specific views, source-grounded answers, a one-page architecture, and a live demonstration.

Non-negotiable boundaries: use only synthetic/public data; access no H-FARM production systems; execute actions only on mocks; require human approval for external communication, spending, writes to systems of record, and access changes; no real invoices; no monitoring or scoring of people.

7. Definition of success

Technical success: one complete, reproducible, source-grounded and policy-compliant operational loop.
Product validation: evidence that the workflow is useful compared with existing manual or deterministic alternatives.
Long-term direction: expand the shared Brain to additional processes **only after** the first loop demonstrates value.

References: H-FARM, *H-OS Challenge Brief* (provided PDF, pp. 1–4); Google Gemma 4 model card; Ollama Gemma 4 E2B. The architecture, ontology, technology choices, and example workflow are this project's design proposals, not statements about H-FARM's internal infrastructure.